

TÀI LIỆU EDUFRAMEWORK

HƯỚNG DẪN PHÁT TRIỂN

Thiết lập môi trường phát triển và tạo dự án EduFramework

PlatformIO · S32K144 · MaaZEDU

Mục lục

1	Giới thiệu	2
2	Thiết lập môi trường phát triển	3
2.1	Cài đặt Visual Studio Code	3
2.2	Cài đặt và cấu hình Git	3
2.3	Cài đặt PlatformIO IDE	7
3	Tạo project EduFramework đầu tiên	8
3.1	Tạo cấu trúc project	8
3.2	Mở project bằng PlatformIO	9
3.3	Cấu hình project	10
3.4	Viết chương trình đầu tiên	11
3.5	Build project	12
3.6	Nạp chương trình lên bo mạch	12
4	Debugging	13
4.1	Bắt đầu phiên debug	13
4.2	Thanh điều khiển và breakpoint	14
4.3	Theo dõi biến với Watch	15
4.4	Tiếp tục chương trình và quan sát kết quả	15
5	Xử lý sự cố	16
6	Bước tiếp theo	16
7	Tài liệu tham khảo	17

1 Giới thiệu

EduFramework là một hệ sinh thái phát triển phần mềm dành cho vi điều khiển NXP S32K144, được xây dựng nhằm đơn giản hóa quá trình phát triển ứng dụng thông qua các API ở mức cao hơn và khả năng tích hợp với PlatformIO. Hệ thống hình thành một quy trình thống nhất từ tổ chức project và phát triển mã nguồn đến build, nạp chương trình và debug trên phần cứng.

Tài liệu này hướng dẫn từng bước quá trình thiết lập môi trường phát triển và tạo một project EduFramework trên bo mạch MaaZEDU. Sau khi hoàn thành các bước thiết lập, môi trường có thể được sử dụng để tiếp tục với EduFramework Laboratory Series hoặc phát triển các ứng dụng riêng trên S32K144.

Hệ sinh thái EduFramework được tổ chức thành ba thành phần chính: **EduFramework**, **Platform-NXPS32K** và **EduFramework Laboratory Series**. Mỗi thành phần đảm nhiệm một vai trò riêng trong quy trình phát triển.

Thành phần	Vai trò
EduFramework	Lớp phần mềm dành cho phát triển ứng dụng, bao gồm các API theo phong cách Arduino, định nghĩa chân logic và thư viện thiết bị cho S32K144.
Platform-NXPS32K	Tích hợp S32K144 và EduFramework vào PlatformIO, hỗ trợ cấu hình project, build, upload và debug.
EduFramework Laboratory Series	Hệ thống bài thực hành, project mẫu và tài liệu hướng dẫn khai thác EduFramework theo từng chủ đề.

Bảng 1.1. Các thành phần chính của hệ sinh thái EduFramework

EduFramework là thành phần phần mềm được ứng dụng sử dụng trực tiếp. Các API theo phong cách Arduino đơn giản hóa những thao tác phổ biến trên vi điều khiển, trong khi các thư viện thiết bị hỗ trợ làm việc với các module và cảm biến bên ngoài. Repository của framework cũng chứa mã nguồn các driver tầng thấp và những thành phần cần thiết cho việc nghiên cứu hoặc tiếp tục phát triển framework.

GitHub Repository

EduFramework Project

Platform-NXPS32K đảm nhiệm việc tích hợp S32K144 với hệ sinh thái PlatformIO. Platform xác định board, framework, toolchain và các công cụ liên quan để PlatformIO có thể thực hiện quy trình build, upload và debug cho project EduFramework. Các định nghĩa và cấu hình của platform có thể được tham khảo trực tiếp trong repository của dự án.

GitHub Repository

Platform-NXPS32K

EduFramework Laboratory Series xây dựng các bài thực hành theo từng chủ đề dựa trên EduFramework và Platform-NXPS32K. Các bài lab được thiết kế để từng bước làm quen với API của framework và áp dụng chúng vào phần cứng, từ đó tạo nền tảng cho việc phát triển các ứng dụng riêng trên S32K144. Repository của Laboratory Series lưu trữ các project thực hành, tài liệu hoàn chỉnh và mã nguồn dùng để phát triển tài liệu.

GitHub Repository

EduFramework Labs and Documentation

2 Thiết lập môi trường phát triển

Trước khi tạo project EduFramework, cần chuẩn bị các công cụ phát triển trên máy tính. Môi trường được sử dụng trong tài liệu gồm Visual Studio Code, Git và PlatformIO IDE. Phần này hướng dẫn cài đặt và kiểm tra từng thành phần trước khi bắt đầu tạo project.

2.1 Cài đặt Visual Studio Code

Visual Studio Code (VS Code) được sử dụng làm trình soạn thảo mã nguồn và là môi trường để tích hợp PlatformIO trong quá trình phát triển với EduFramework.

Truy cập trang chính thức của Visual Studio Code:

GitHub Repository

Visual Studio Code

Tải phiên bản Visual Studio Code phù hợp với hệ điều hành đang sử dụng và thực hiện cài đặt theo hướng dẫn trên trang chính thức. Sau khi hoàn tất, khởi động Visual Studio Code để xác nhận ứng dụng hoạt động bình thường.

Ghi chú

Giao diện và quá trình cài đặt Visual Studio Code có thể khác nhau tùy theo hệ điều hành và phiên bản phần mềm. Development Guide không yêu cầu thay đổi cấu hình cài đặt đặc biệt của Visual Studio Code.

2.2 Cài đặt và cấu hình Git

Git được sử dụng để quản lý mã nguồn và cho phép PlatformIO truy cập các thành phần của EduFramework được lưu trữ trên GitHub. Ngoài việc cài đặt Git, cần thiết lập thông tin người dùng để Git có thể ghi nhận tác giả của các commit được tạo trên máy tính.

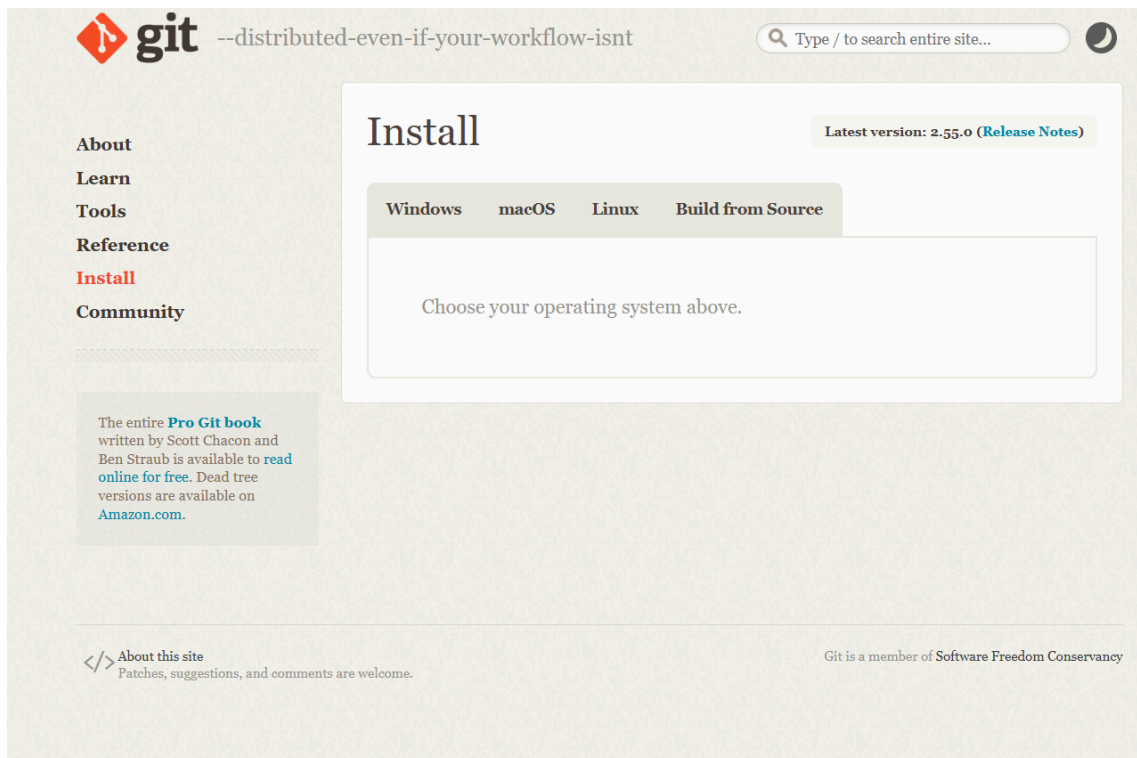
2.2.1 Tải và cài đặt Git

Truy cập trang cài đặt chính thức của Git:

GitHub Repository

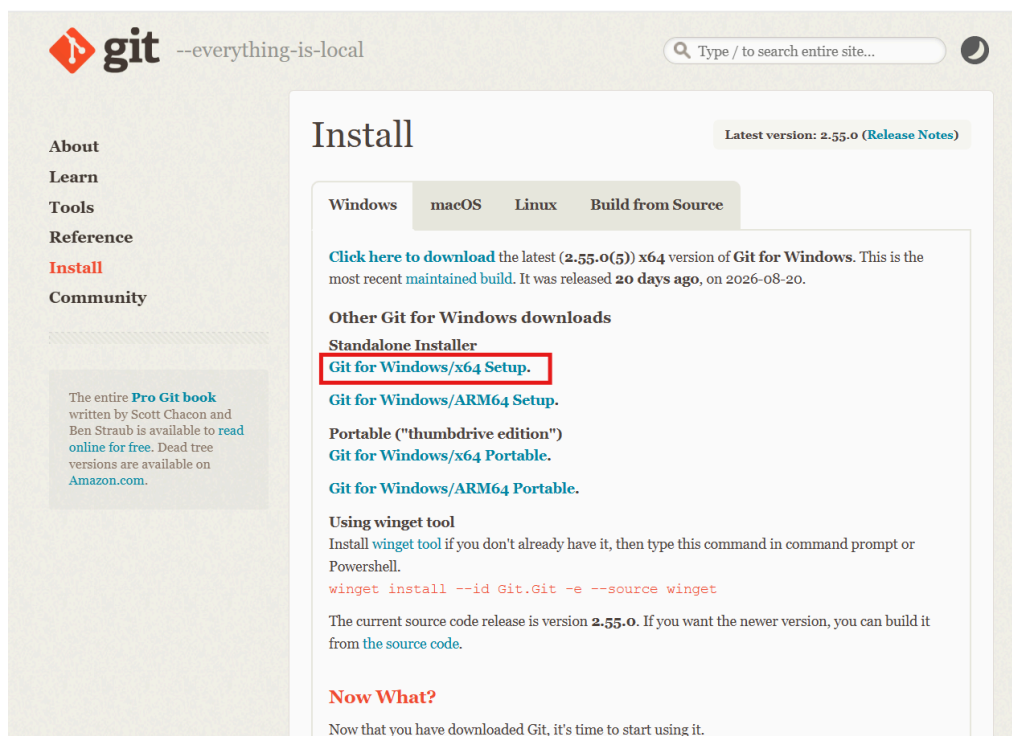
Git - Install

Tại trang cài đặt, chọn phiên bản Git phù hợp với hệ điều hành đang sử dụng.



Hình 2.1. Trang cài đặt chính thức của Git

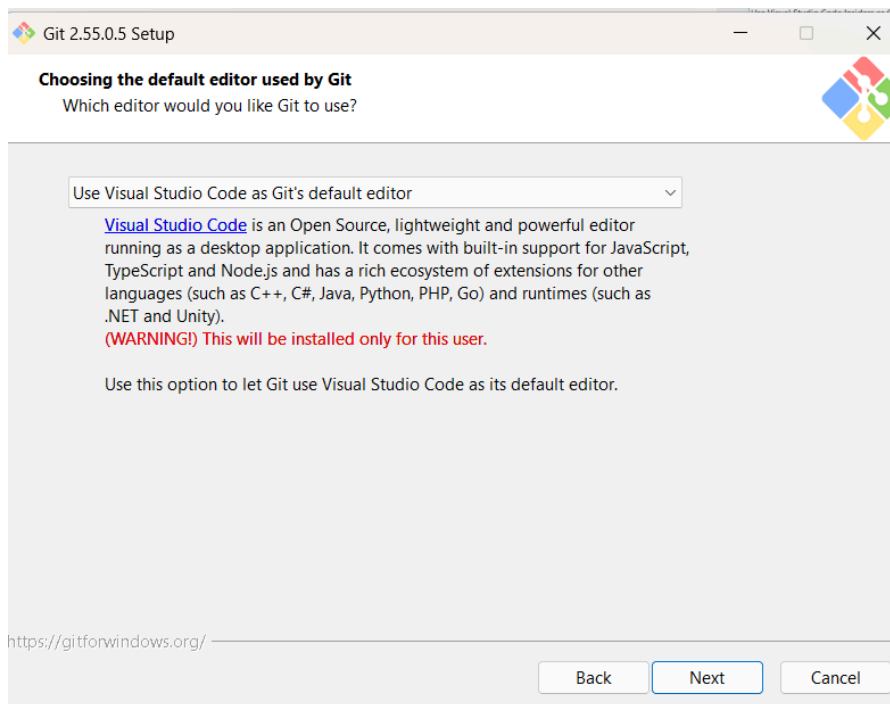
Sau khi lựa chọn hệ điều hành, tải bộ cài phù hợp với hệ thống. Hình dưới đây minh họa quá trình lựa chọn bộ cài Git trên Windows x64.



Hình 2.2. Ví dụ lựa chọn bộ cài Git trên Windows

Thực hiện cài đặt Git bằng bộ cài vừa tải xuống. Các tùy chọn mặc định của trình cài đặt có thể được giữ nguyên nếu không có yêu cầu cấu hình riêng.

Đối với Git for Windows, khi trình cài đặt yêu cầu lựa chọn trình soạn thảo mặc định của Git, chọn **Visual Studio Code** để sử dụng VS Code cho các thao tác Git cần mở trình soạn thảo.



Hình 2.3. Lựa chọn Visual Studio Code làm trình soạn thảo mặc định của Git

Tiếp tục quá trình cài đặt với các tùy chọn mặc định cho đến khi hoàn tất.

2.2.2 Kiểm tra cài đặt Git

Sau khi cài đặt, mở terminal và thực hiện lệnh:

```
Terminal  
git --version
```

Nếu Git đã được cài đặt và có thể được truy cập từ terminal, kết quả sẽ hiển thị phiên bản Git hiện có trên hệ thống.

```
PS C:\Users\Asus> git --version  
git version 2.55.0.windows.5
```

Hình 2.4. Kiểm tra phiên bản Git sau khi cài đặt

Ghi chú

Số phiên bản hiển thị có thể khác với hình minh họa. Chỉ cần lệnh `git --version` trả về thông tin phiên bản hợp lệ là quá trình cài đặt Git đã thành công.

2.2.3 Cấu hình thông tin người dùng Git

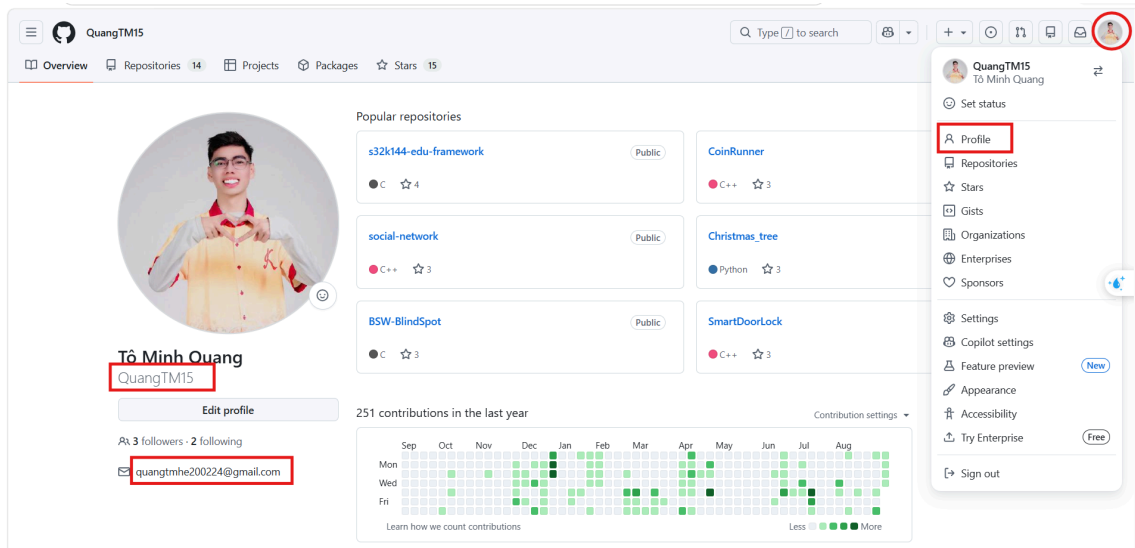
Git sử dụng `user.name` và `user.email` để xác định thông tin tác giả được ghi trong mỗi commit. Nếu sử dụng GitHub để lưu trữ repository, nên sử dụng địa chỉ email đã được liên kết với tài khoản GitHub để các commit có thể được nhận diện đúng với tài khoản tương ứng.

Nếu chưa có tài khoản GitHub, có thể tạo tài khoản tại trang chính thức:

GitHub Repository

GitHub

Thông tin tài khoản và địa chỉ email có thể được kiểm tra trong GitHub trước khi cấu hình Git. Hình dưới đây minh họa vị trí thông tin tài khoản trên GitHub.



Hình 2.5. Ví dụ thông tin tài khoản và địa chỉ email trên GitHub

Mở terminal và thiết lập tên được sử dụng cho các commit bằng lệnh:

Terminal

```
git config --global user.name "Your Name"
```

Tiếp theo, thiết lập địa chỉ email:

Terminal

```
git config --global user.email "your-email@example.com"
```

Trong đó, `Your Name` là tên được ghi nhận trong commit và `your-email@example.com` là địa chỉ email của người dùng. Giá trị `user.name` không bắt buộc phải giống tên tài khoản GitHub.

Tùy chọn `--global` áp dụng các giá trị trên cho các repository Git của tài khoản người dùng hiện tại trên máy tính.

Sau khi cấu hình, kiểm tra lại thiết lập bằng lệnh:

Terminal

```
git config --global --list
```

Kết quả phải chứa các thông tin `user.name` và `user.email` đã thiết lập.

Git global configuration

```
user.name=Your Name user.email=your-email@example.com
```

Hình dưới đây minh họa kết quả sau khi Git đã được cấu hình thành công.

```
PS C:\Users\Asus> git config --global --list
core.editor="C:\Users\Asus\AppData\Local\Programs\Microsoft VS Code\bin\code" --wait
user.name=QuangTM15
user.email=quangtmhe200224@gmail.com
filter.lfs.required=true
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
PS C:\Users\Asus>
```

Hình 2.6. Kiểm tra cấu hình người dùng Git

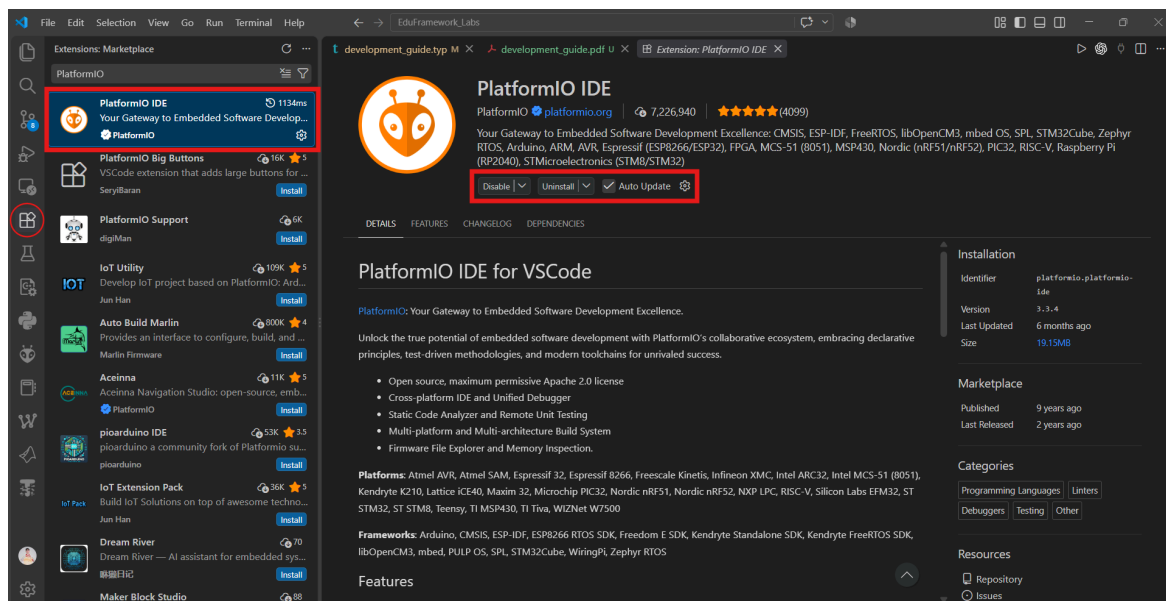
KẾT QUẢ MONG ĐỢI

Git đã được cài đặt và cấu hình thành công. Lệnh `git --version` có thể được thực thi từ terminal, đồng thời kết quả của `git config --global --list` chứa `user.name` và `user.email` của người dùng.

2.3 Cài đặt PlatformIO IDE

PlatformIO IDE được tích hợp vào Visual Studio Code thông qua extension chính thức của PlatformIO. Extension này cung cấp môi trường phát triển cho các project embedded và sẽ được sử dụng để build, upload và debug các project EduFramework.

Mở Visual Studio Code, chọn **Extensions** trên Activity Bar và tìm kiếm **PlatformIO IDE**. Chọn extension **PlatformIO IDE** do **PlatformIO** phát hành, sau đó chọn **Install** để bắt đầu cài đặt.



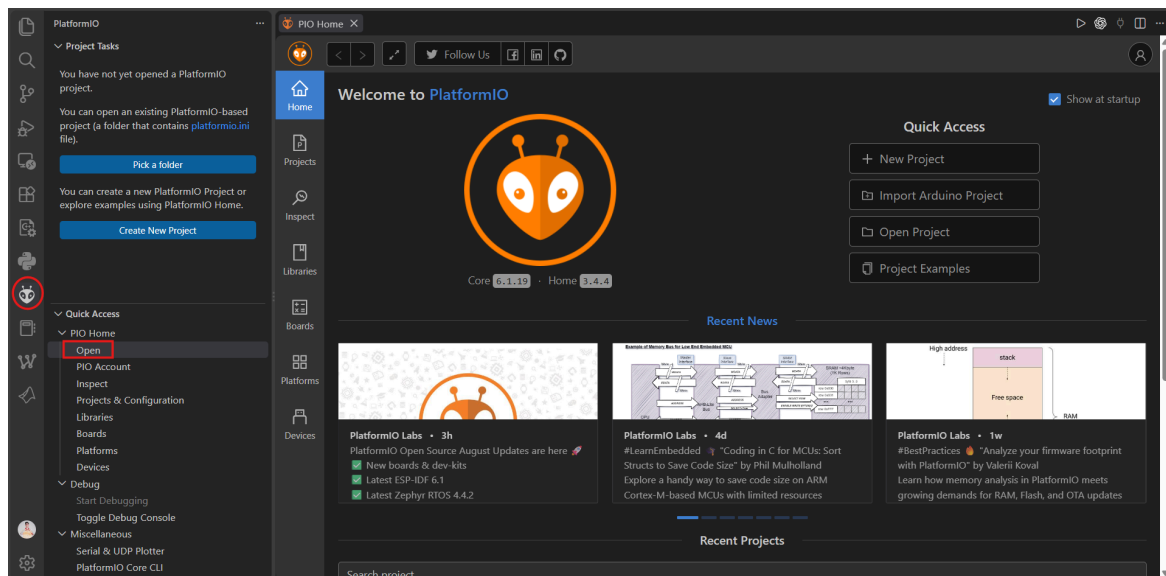
Hình 2.7. Cài đặt PlatformIO IDE từ Visual Studio Code Extensions Marketplace

Sau khi cài đặt extension, PlatformIO sẽ thực hiện quá trình khởi tạo các thành phần cần thiết. Quá trình này có thể mất một khoảng thời gian trong lần khởi động đầu tiên.

Ghi chú

Trong lần khởi động đầu tiên, cần chờ PlatformIO hoàn tất quá trình khởi tạo trước khi tiếp tục. Thời gian thực hiện có thể khác nhau tùy thuộc vào hệ thống và kết nối mạng.

Sau khi quá trình khởi tạo hoàn tất, chọn biểu tượng **PlatformIO** trên Activity Bar, sau đó chọn **PIO Home** → **Open** để mở PlatformIO Home.



Hình 2.8. Giao diện PlatformIO Home sau khi cài đặt thành công

KẾT QUẢ MONG ĐỢI

PlatformIO IDE đã được cài đặt và khởi tạo thành công. PlatformIO Home có thể được mở trực tiếp từ Visual Studio Code.

3 Tạo project EduFramework đầu tiên

Phần này hướng dẫn tạo một project EduFramework tối thiểu trên S32K144, cấu hình project với PlatformIO, viết chương trình đầu tiên và nạp chương trình lên bo mạch MaaZEDU.

3.1 Tạo cấu trúc project

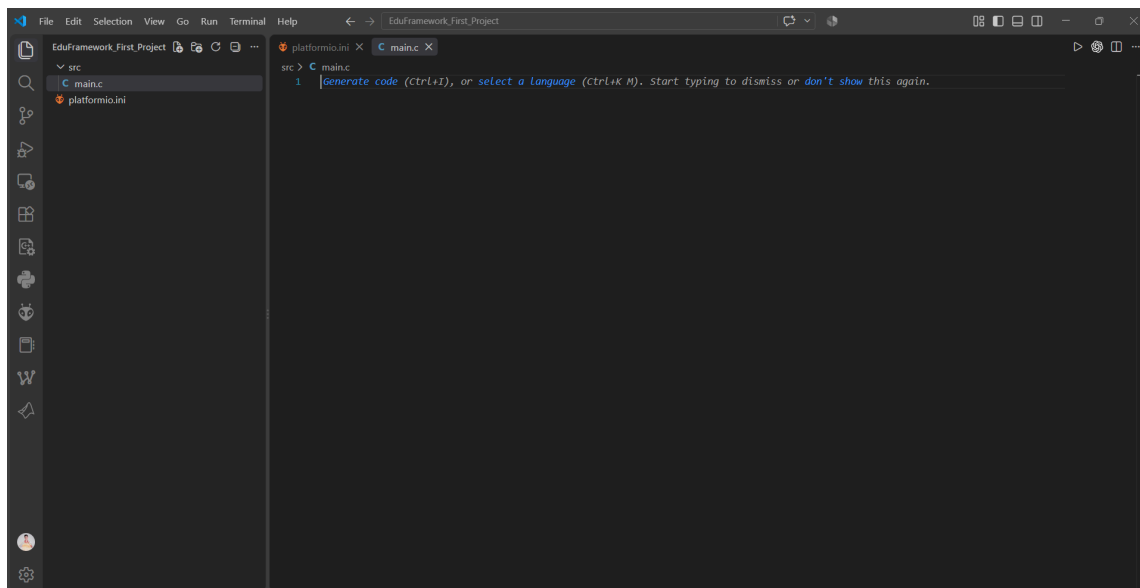
Trong Visual Studio Code, chọn **File** → **Open Folder....** Tạo một thư mục mới cho project, ví dụ `EduFramework_First_Project`, sau đó mở thư mục vừa tạo trong Visual Studio Code.

Trong Explorer của Visual Studio Code, tạo thư mục `src`, sau đó tạo file `main.c` bên trong thư mục này. Tại thư mục gốc của project, tạo thêm file `platformio.ini`.

Cấu trúc project ban đầu như sau:

Cấu trúc project EduFramework

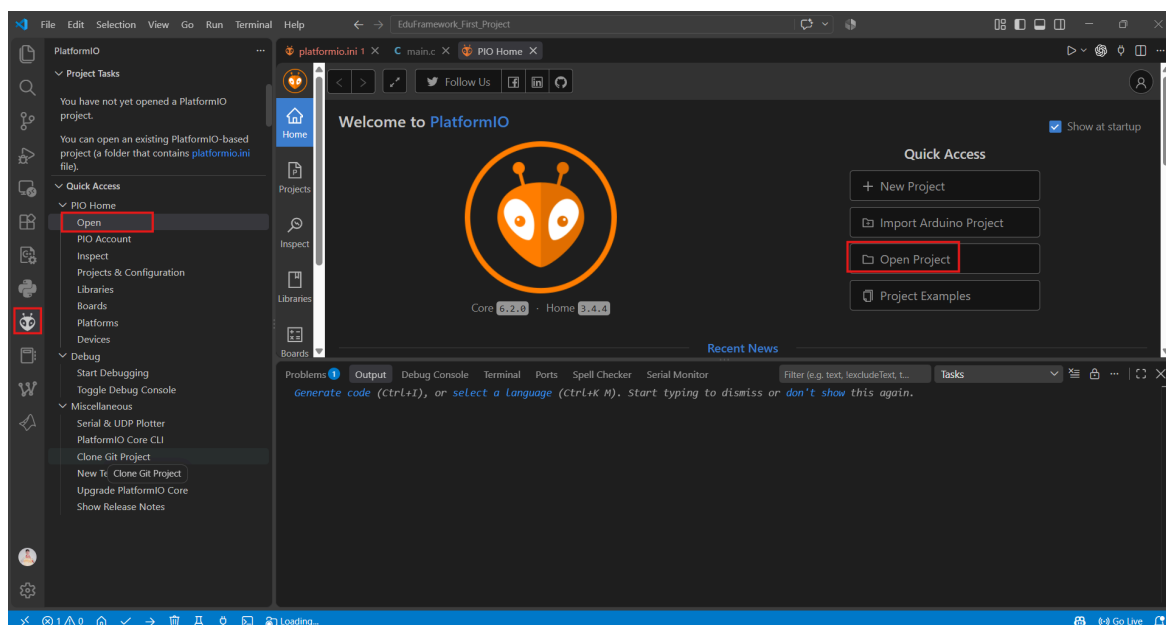
```
EduFramework_First_Project/
├── platformio.ini
└── src/
    └── main.c
```



Hình 3.1. Cấu trúc ban đầu của project EduFramework

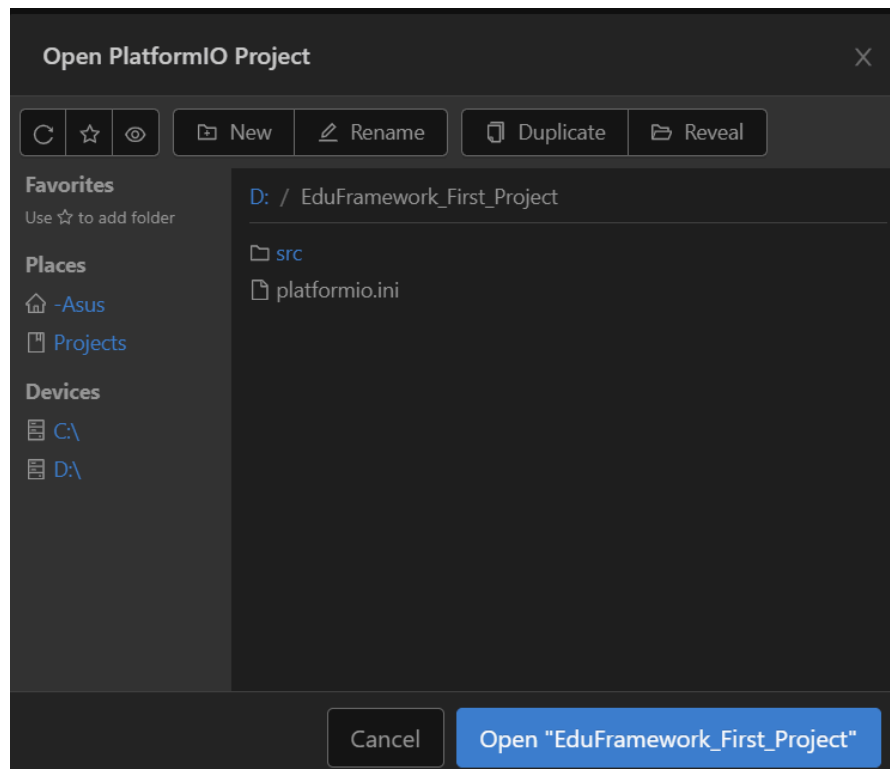
3.2 Mở project bằng PlatformIO

Sau khi tạo cấu trúc project, mở PlatformIO Home bằng biểu tượng PlatformIO trên Activity Bar. Chọn **PIO Home** → **Open** → **Open Project** để mở project.



Hình 3.2. Mở project từ PlatformIO Home

Chọn thư mục project vừa tạo. Thư mục được chọn phải chứa file `platformio.ini` và thư mục `src`.



Hình 3.3. Lựa chọn thư mục project EduFramework

3.3 Cấu hình project

Mở file `platformio.ini` và khai báo environment cho S32K144 như sau:

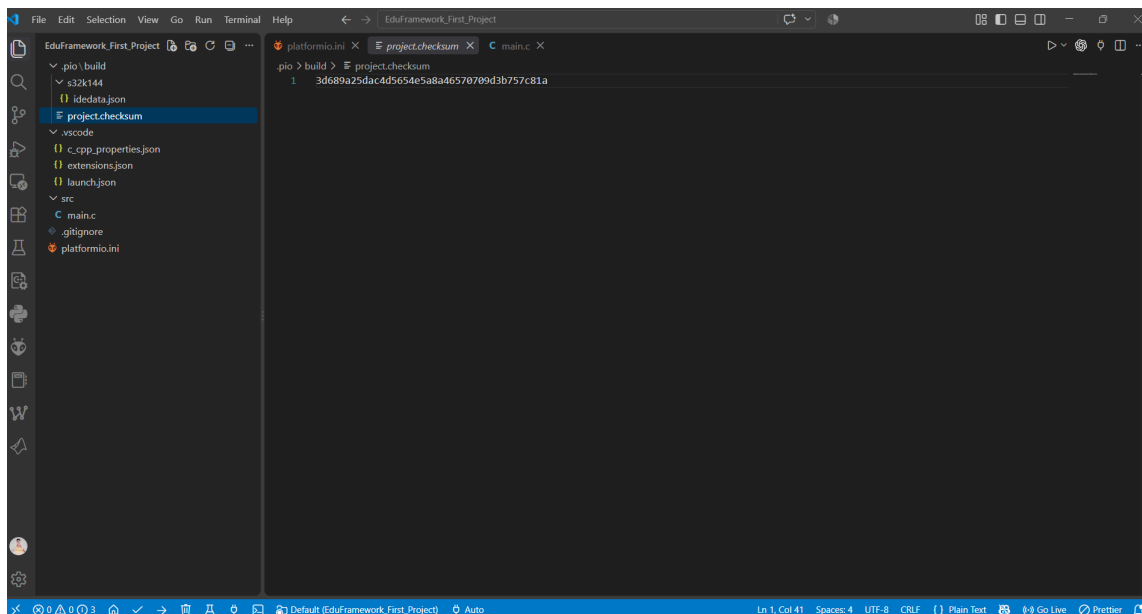
```
[env:s32k144]
platform = https://github.com/QuangTM15/platform-nxps32k.git
board = s32k144
framework = eduframework
upload_protocol = jlink
debug_tool = jlink
```

Mã nguồn 3.1. Cấu hình PlatformIO cho project EduFramework

Trong cấu hình trên, `platform` xác định Platform-NXPS32K được sử dụng bởi project, `board` xác định bo mạch đích, `framework` lựa chọn EduFramework, trong khi `upload_protocol` và `debug_tool` cấu hình J-Link cho quá trình nạp chương trình và debug.

Sau khi lưu `platformio.ini`, PlatformIO sẽ nhận diện cấu hình project và chuẩn bị môi trường phát triển. Trong lần sử dụng đầu tiên, quá trình này có thể mất một khoảng thời gian do PlatformIO cần tải và cài đặt các thành phần cần thiết.

Các thư mục và file do PlatformIO quản lý, chẳng hạn `.pio` và `.vscode`, có thể xuất hiện trong project sau khi quá trình khởi tạo hoàn tất.



Hình 3.4. Project EduFramework sau khi được PlatformIO khởi tạo

Ghi chú

Không cần chỉnh sửa thủ công các file bên trong thư mục `.pio`. Nội dung trong thư mục này được PlatformIO quản lý và có thể được tạo lại trong quá trình build project.

3.4 Viết chương trình đầu tiên

Mở file `src/main.c` và nhập chương trình sau:

```
#include "Arduino.h"

volatile int counter = 0;

int main(void)
{
    setup();

    pinMode(LED_RED, OUTPUT);

    while (1)
    {
        digitalWrite(LED_RED, LOW);
        delay(500U);

        digitalWrite(LED_RED, HIGH);
        delay(500U);

        counter++;
    }

    return 0;
}
```

Mã nguồn 3.2. Chương trình Blink LED đầu tiên với EduFramework

Hàm `setup()` phải được gọi trước khi sử dụng các API của EduFramework. Hàm này thực hiện quá trình khởi tạo tối thiểu các thành phần phần cứng và tài nguyên cần thiết để framework có thể hoạt động trước khi chương trình ứng dụng tiếp tục thực thi.

CẢNH BÁO

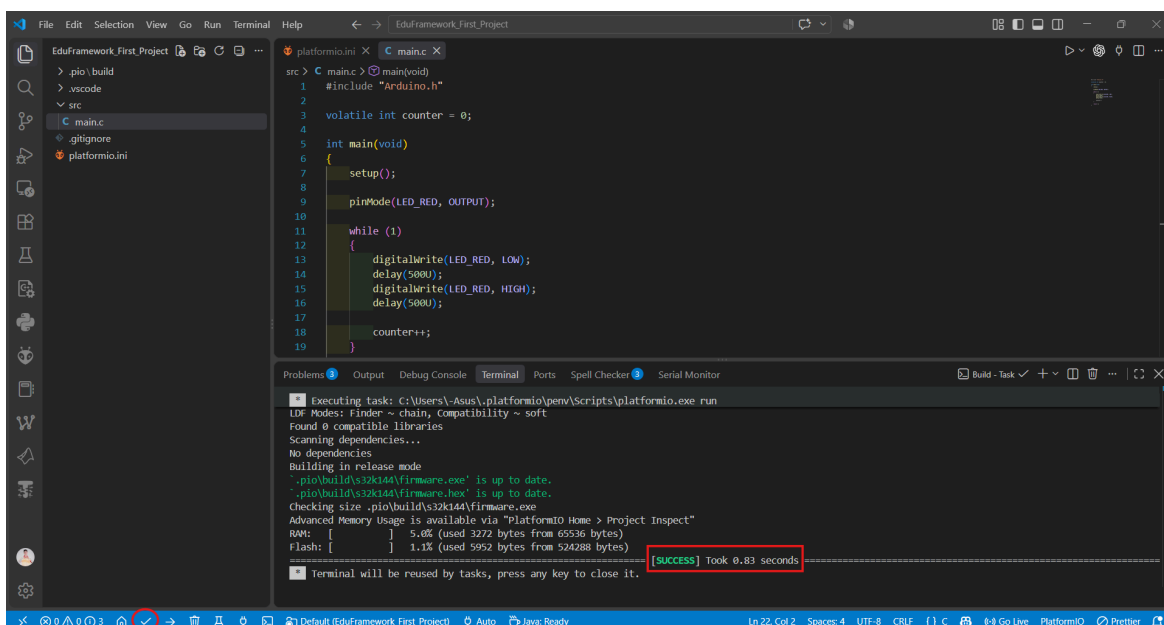
Khi phát triển ứng dụng với EduFramework, bắt buộc phải gọi `setup()` và phải gọi `setup()` trước khi sử dụng bất kỳ API nào của framework. Việc bỏ qua bước khởi tạo này có thể dẫn đến treo hệ thống hoặc các hành vi không mong muốn trong quá trình thực thi chương trình.

3.5 Build project

Trước khi build, lưu toàn bộ các file đã chỉnh sửa.

Chọn biểu tượng **Build** trên thanh công cụ PlatformIO ở phía dưới Visual Studio Code để bắt đầu biên dịch project.

Khi quá trình build hoàn tất thành công, terminal sẽ hiển thị trạng thái **SUCCESS**.

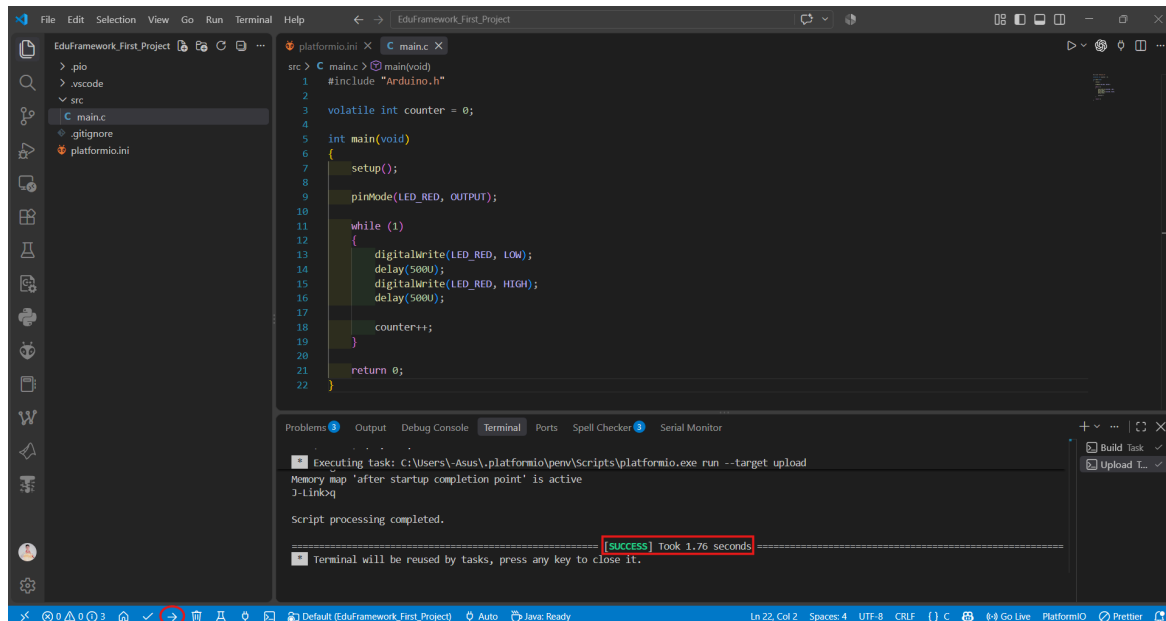


Hình 3.5. Kết quả build project EduFramework thành công

3.6 Nạp chương trình lên bo mạch

Kết nối bo mạch MaaZEDU với máy tính bằng cổng USB được sử dụng cho quá trình nạp chương trình và debug. Sau khi bo mạch đã được kết nối, chọn biểu tượng **Upload** trên thanh công cụ PlatformIO để nạp chương trình lên S32K144.

PlatformIO sử dụng cấu hình `upload_protocol = jlink` trong `platformio.ini` để thực hiện quá trình nạp chương trình. Khi quá trình hoàn tất thành công, terminal sẽ hiển thị trạng thái **SUCCESS**.



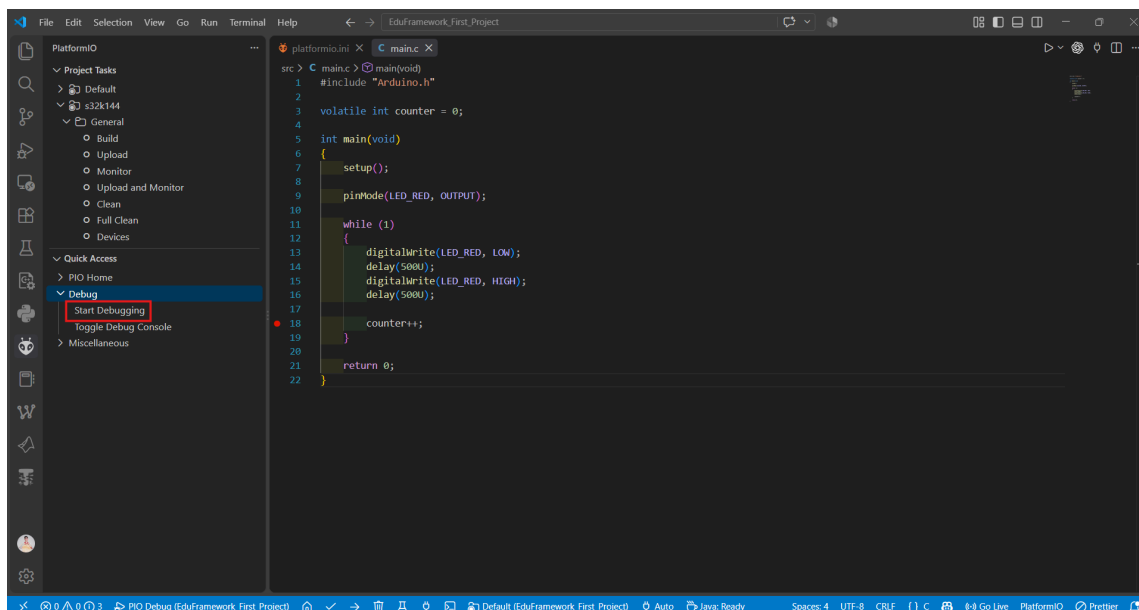
Hình 3.6. Kết quả upload chương trình lên S32K144 thành công

4 Debugging

PlatformIO tích hợp debugger trực tiếp trong Visual Studio Code, cho phép tạm dừng chương trình, thực thi từng bước và theo dõi giá trị của các biến trong quá trình chạy. Phần này sử dụng project đã tạo ở phần trước để thực hiện một phiên debug cơ bản trên S32K144.

4.1 Bắt đầu phiên debug

Đảm bảo bo mạch MaaZEDU vẫn được kết nối với máy tính qua cổng USB dùng cho nạp chương trình và debug. Trong Visual Studio Code, chọn biểu tượng PlatformIO trên Activity Bar. Trong mục **Debug**, chọn **Start Debugging** để bắt đầu phiên debug.



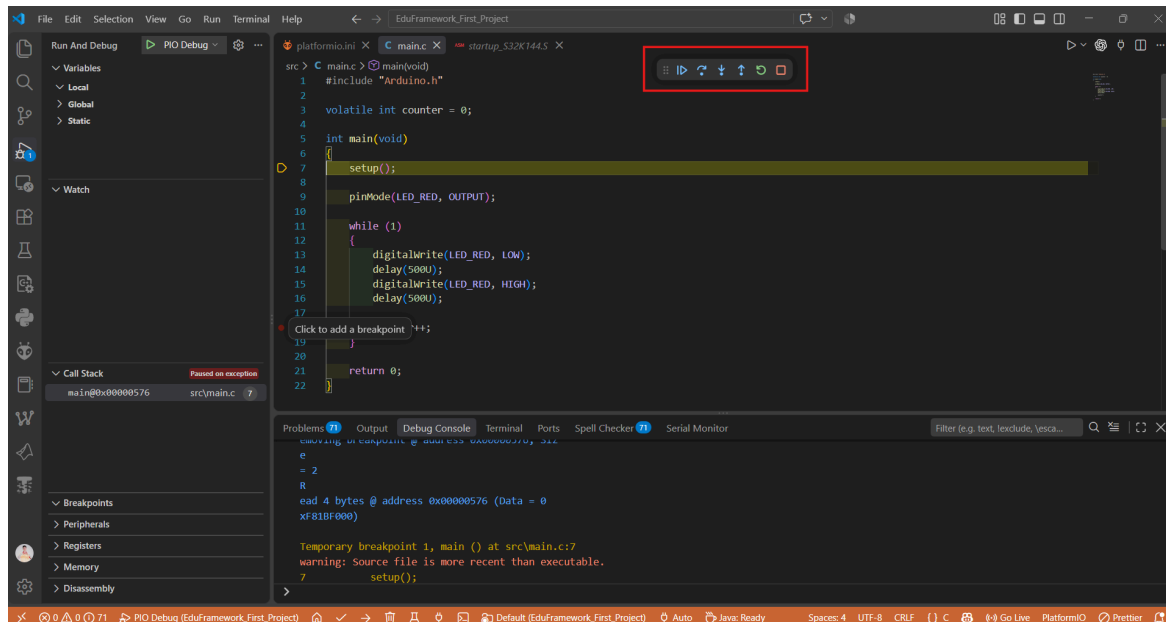
Hình 4.1. Bắt đầu phiên debug từ PlatformIO

PlatformIO sẽ chuẩn bị chương trình, kết nối với debugger và khởi động phiên debug. Sau khi quá trình hoàn tất, Visual Studio Code chuyển sang giao diện **Run and Debug**.

Trong cấu hình hiện tại, chương trình có thể tạm dừng tại đầu hàm `main()` khi phiên debug bắt đầu. Từ thời điểm này, có thể sử dụng các công cụ debug để điều khiển quá trình thực thi chương trình.

4.2 Thanh điều khiển và breakpoint

Khi phiên debug đang hoạt động, thanh điều khiển debug xuất hiện ở phía trên cửa sổ Visual Studio Code.



Hình 4.2. Giao diện debug, thanh điều khiển và breakpoint

Các nút trên thanh điều khiển được sắp xếp từ trái sang phải như sau:

Công cụ	Phím tắt	Chức năng
Continue	F5	Tiếp tục chạy chương trình cho đến khi gặp breakpoint tiếp theo hoặc một điều kiện làm chương trình dừng lại.
Step Over	F10	Thực thi dòng hiện tại và chuyển sang dòng tiếp theo mà không đi vào bên trong hàm được gọi tại dòng đó.
Step Into	F11	Đi vào bên trong hàm được gọi tại dòng hiện tại để tiếp tục debug từng lệnh bên trong hàm.
Step Out	Shift + F11	Tiếp tục chạy cho đến khi thoát khỏi hàm hiện tại và quay lại hàm gọi.
Restart	Ctrl + Shift + F5	Khởi động lại phiên debug từ đầu.
Stop	Shift + F5	Kết thúc phiên debug hiện tại.

Bảng 4.1. Các công cụ điều khiển debug cơ bản

Breakpoint là một điểm dừng được đặt tại một dòng mã nguồn. Khi chương trình chạy tới dòng có breakpoint, debugger sẽ tạm dừng quá trình thực thi để có thể quan sát biến và trạng thái chương trình.

Để đặt breakpoint, nhấp vào vùng lề bên trái số dòng cần dừng. Một dấu tròn màu đỏ xuất hiện tại dòng đó. Nhấp lại vào cùng vị trí để bỏ breakpoint.

Trong project hiện tại, đặt breakpoint tại dòng:

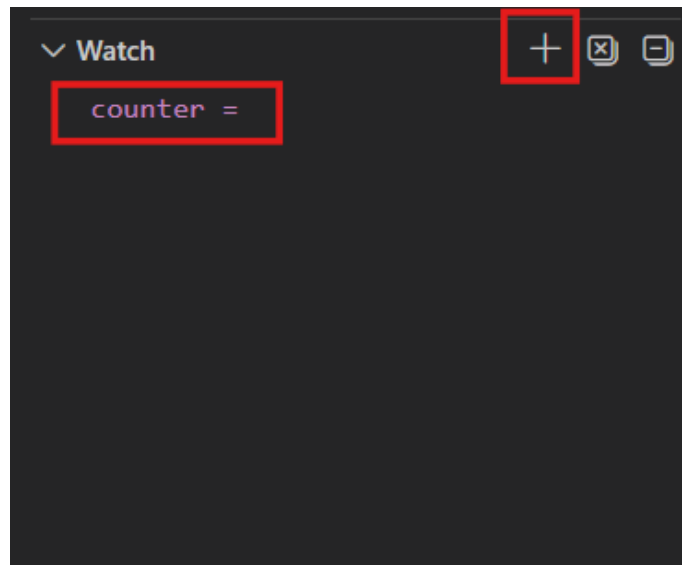
```
counter++;
```

Mã nguồn 4.1. Dòng lệnh được sử dụng làm breakpoint

4.3 Theo dõi biến với Watch

Cửa sổ **Watch** cho phép theo dõi trực tiếp giá trị của một biến hoặc biểu thức trong quá trình debug. Trong giao diện **Run and Debug**, mở mục **Watch**, chọn biểu tượng **+** và nhập:

```
Terminal  
counter
```



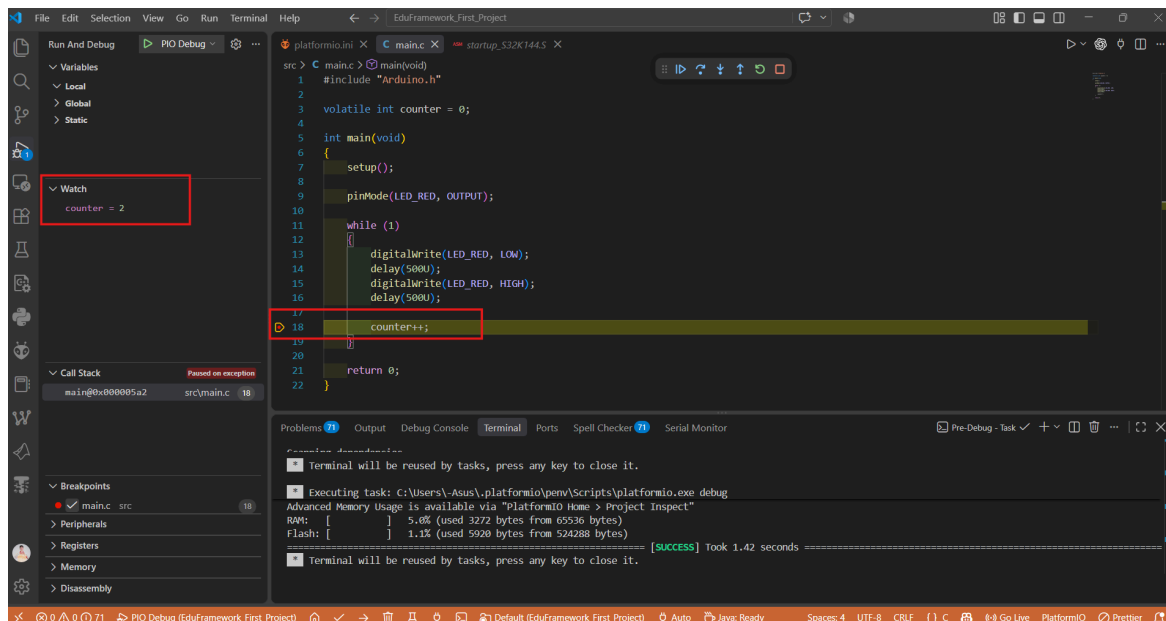
Hình 4.3. Thêm biến counter vào cửa sổ Watch

Sau khi được thêm vào Watch, giá trị của `counter` sẽ được cập nhật mỗi khi chương trình tạm dừng trong phiên debug.

4.4 Tiếp tục chương trình và quan sát kết quả

Sau khi đặt breakpoint tại `counter++` và thêm biến `counter` vào Watch, chọn **Continue** (F5) để tiếp tục chạy chương trình.

Chương trình sẽ thực thi cho đến khi gặp breakpoint tại dòng `counter++`, sau đó debugger sẽ tạm dừng tại vị trí này. Giá trị hiện tại của `counter` có thể được quan sát trong cửa sổ Watch.



Hình 4.4. Quan sát giá trị counter tại breakpoint

Tiếp tục chọn **Continue** (F5) để chương trình chạy qua một chu kỳ tiếp theo. Khi breakpoint được kích hoạt lại, giá trị của `counter` thay đổi theo quá trình thực thi chương trình.

Các công cụ **Step Over**, **Step Into** và **Step Out** có thể được sử dụng khi cần quan sát chi tiết hơn luồng thực thi của chương trình tại vị trí đang dừng.

5 Xử lý sự cố

Phần này sẽ được hoàn thiện sau quá trình kiểm thử toàn bộ quy trình thiết lập và phát triển EduFramework trên một môi trường máy tính mới. Các vấn đề thực tế liên quan đến cài đặt công cụ, khởi tạo PlatformIO, build, upload và debug sẽ được tổng hợp cùng với hướng xử lý tương ứng.

6 Bước tiếp theo

Sau khi hoàn thành các bước trong tài liệu này, môi trường phát triển EduFramework đã sẵn sàng để xây dựng, nạp và debug các ứng dụng trên S32K144 thông qua PlatformIO.

EduFramework Laboratory Series tiếp tục giới thiệu các chức năng của framework thông qua hệ thống bài thực hành theo từng chủ đề. Các bài thực hành được xây dựng từ những chức năng cơ bản như Digital Output và Digital Input, sau đó mở rộng sang các ngoại vi và thiết bị được hỗ trợ bởi EduFramework.

Toàn bộ tài liệu thực hành, project mẫu và mã nguồn liên quan được duy trì tại:

GitHub Repository

EduFramework Laboratory Series

Các thành phần của EduFramework và Platform-NXPS32K có thể được tham khảo trực tiếp tại các repository tương ứng:

GitHub Repository

EduFramework

GitHub Repository

Platform-NXPS32K

7 Tài liệu tham khảo

- [1] Git, *Git Documentation* .
- [2] GitHub, *GitHub Docs* .
- [3] Microsoft, *Visual Studio Code Documentation* .
- [4] PlatformIO, *PlatformIO Documentation* .
- [5] QuangTM15, *EduFramework for NXP S32K144* .
- [6] QuangTM15, *Platform-NXPS32K* .
- [7] QuangTM15, *EduFramework Laboratory Series* .